Programming Tools for Scientific Computing Final Project Report



TECHNISCHE UNIVERSITÄT DARMSTADT

Akhil Unnikrishnan
July 2026

Contents

1 Technical Background **3**
- 1.1 Introduction . . . 3
- 1.2 Numerical Background . . . 3
- 1.3 Eigen Library . . . 3
- 1.4 CMake . . . 3
- 1.5 Git and GitLab . . . 4
- 1.6 Docker . . . 4
- 1.7 Bash Scripting . . . 4
- 1.8 VTK and ParaView . . . 4

2 Project Execution **6**
- 2.1 Project Setup . . . 6
- 2.2 Laplacian Matrix Assembly . . . 6
- 2.3 Explicit Time Integration . . . 8
- 2.4 Implicit Time Integration . . . 8
- 2.5 Output Generation and Visualization . . . 9
- 2.6 Bash Automation . . . 12
- 2.7 Docker Environment . . . 12
- 2.8 Challenges Encountered and Solutions . . . 13
  - 2.8.1 Challenge 1 - Sparse Laplacian Assembly . . . 13
  - 2.8.2 Solution . . . 13
  - 2.8.3 Challenge 2 - Implicit Euler and Newton Method . . . 13
  - 2.8.4 Solution . . . 13
  - 2.8.5 Challenge 3 - Docker and Dependencies . . . 13
  - 2.8.6 Solution . . . 13
  - 2.8.7 Challenge 4 - ParaView in Docker . . . 13
  - 2.8.8 Solution . . . 14
  - 2.8.9 Challenge 5 - Automation . . . 14
  - 2.8.10 Solution . . . 14
- 2.9 Lessons Learned and Future Improvements . . . 14
- 2.10 Results . . . 15
- 2.11 Effect of the Model Parameter C_b . . . 15

2.12 Validation with the Reference Solution 15
2.13 Effect of Time Step Size 15
2.14 Quantitative Error Analysis 17
2.15 Effect of Time Step Size 17
2.16 Discussions 17
2.17 Conclusion 17

1 Technical Background

1.1 Introduction

The objective of this project was to develop a numerical solver for the two-dimensional Brusselator reaction–diffusion system as part of the *Programming Tools for Scientific Computing* course at Technische Universität Darmstadt. The governing equations were spatially discretized using the finite difference method and solved using both explicit and implicit time integration schemes. In addition to the numerical implementation, the project emphasized the use of modern scientific computing tools and workflows, including C++, Eigen, CMake, Bash scripting, Docker, GitLab CI/CD, and VTK for visualization. This report summarizes the technical background of the tools used, discusses the implementation process and challenges encountered during development, and finally presents the results obtained from various parameter studies.

1.2 Numerical Background

The Brusselator is a reaction–diffusion model that describes the interaction of two chemical species undergoing diffusion and nonlinear chemical reactions. It consists of two coupled, time-dependent partial differential equations, where the evolution of the concentrations is governed by both diffusion and reaction source terms. Due to the nonlinear nature of the governing equations, analytical solutions are generally not available, making numerical methods necessary.

In this project, the governing equations were spatially discretized using the finite difference method (FDM) on a structured two-dimensional grid. Central differencing was employed to approximate the spatial derivatives, while homogeneous Neumann boundary conditions were enforced to prevent diffusive flux across the domain boundaries. The spatial discretization transformed the governing partial differential equations into a system of ordinary differential equations.

The resulting system was integrated in time using both explicit and implicit numerical schemes. The explicit Runge–Kutta method was implemented to provide an efficient and straightforward solution procedure for sufficiently small time steps. In addition, the implicit Backward Euler method was implemented to improve numerical stability for larger time steps. Since the implicit formulation leads to a nonlinear algebraic system at every time step, an inexact Newton method was employed to iteratively obtain the solution. The Jacobian matrix and sparse linear algebra routines from the Eigen library were used to efficiently solve the resulting linear systems.

1.3 Eigen Library

The Eigen library is a high-performance, header-only C++ library for linear algebra and numerical computations. It provides efficient data structures and algorithms for operations involving dense and sparse matrices, vectors, and linear system solvers. Due to its ease of integration and extensive functionality, Eigen was selected as the primary numerical library for this project.

The finite difference discretization of the Brusselator equations results in large sparse matrices, where only a small number of entries are non-zero. To efficiently store and manipulate these matrices, the `Eigen::SparseMatrix` class was used. The discrete Laplacian operator was assembled using triplet lists (`Eigen::Triplet`) before being converted into compressed sparse matrix format.

For the implicit Backward Euler solver, the resulting sparse linear systems arising during the Newton iterations were solved using Eigen's sparse direct solvers. The library provided efficient matrix factorization and solution routines while maintaining a straightforward interface for implementation. By utilizing Eigen's sparse matrix capabilities, the solver achieved both computational efficiency and clean, maintainable code. The use of sparse matrix representations significantly reduced memory consumption and computational cost compared to dense matrix storage, making the implementation suitable for larger grid resolutions.

1.4 CMake

CMake is a cross-platform build system generator that simplifies the process of compiling and managing software projects. Instead of manually specifying compiler commands and dependencies, CMake generates platform-specific build files from a single configuration file (`CMakeLists.txt`). This provides a consistent and reproducible build process across different development environments.

In this project, CMake was used to configure the compilation of the numerical solver, manage external dependencies such as the Eigen library and VTK, and generate the executable files. The modular structure of the project allowed individual source files

to be organized efficiently while ensuring that all required libraries were correctly linked during compilation. The build system also supported the modular organization of the project by compiling source files from separate directories into a single executable.

By automating the build process, CMake reduced the complexity of project management and improved maintainability. Any modifications to the source code or project structure could be incorporated by simply regenerating the build files, allowing the development effort to focus on the implementation of numerical methods rather than build configuration.

1.5 Git and GitLab

Git is a distributed version control system that enables efficient tracking and management of source code throughout the software development process. It records changes to the project, allowing modifications to be reviewed, reverted, or extended while maintaining a complete history of the implementation. GitLab complements Git by providing a remote repository for code hosting, collaboration, and automated development workflows.

Throughout this project, Git was used to manage the development of the numerical solver by organizing changes into meaningful commits and maintaining version consistency. GitLab served as the central repository for synchronizing the project across development environments and preserving the complete project history. The use of version control facilitated incremental implementation, debugging, and testing without compromising previously verified functionality.

Furthermore, GitLab CI/CD was integrated into the project to automatically build and validate the source code whenever changes were pushed to the repository. This continuous integration workflow ensured that new modifications remained compatible with the existing codebase and helped identify compilation or configuration issues at an early stage. Together, Git and GitLab contributed to a reliable, reproducible, and well-structured software development process.

1.6 Docker

Docker is a containerization platform that enables applications to be developed and executed within isolated, portable environments known as containers. By packaging the application together with all required libraries, dependencies, and system configurations, Docker ensures that the software behaves consistently across different operating systems and development environments.

In this project, Docker was used to provide a reproducible environment for compiling and executing the numerical solver. The development environment included all necessary tools and libraries, such as the C++ compiler, CMake, Eigen, and VTK, eliminating the need for manual installation and configuration on the host system.

Furthermore, Docker facilitated integration with the GitLab CI/CD pipeline, allowing the project to be automatically built and tested in a standardized environment whenever changes were pushed to the repository. By isolating the software dependencies from the host operating system, Docker improved the portability, reproducibility, and reliability of the overall development workflow. The use of Docker also enhanced the reproducibility of numerical experiments by ensuring that the simulations were executed in an identical software environment, independent of the underlying hardware or operating system.

1.7 Bash Scripting

Bash is a command-line scripting language widely used in Linux-based operating systems to automate repetitive tasks and manage software workflows. By combining shell commands into executable scripts, Bash enables complex sequences of operations to be performed efficiently without manual intervention.

In this project, Bash scripts were developed to automate the execution of numerical simulations with different parameter configurations. Instead of manually modifying input files and running the solver multiple times, the scripts systematically varied simulation parameters, executed the solver, and organized the generated output files into dedicated directories. This significantly reduced the effort required to perform parameter studies and improved the consistency of the simulation workflow.

The use of Bash scripting also simplified project management by streamlining repetitive tasks such as compiling the project, running simulations, and handling output files. As a result, the overall development process became more efficient, reproducible, and less prone to human error.

1.8 VTK and ParaView

The Visualization Toolkit (VTK) is an open-source software library for representing and processing scientific data, while ParaView is a visualization application built upon VTK for the analysis and interpretation of simulation results. Together, they provide an efficient framework for storing numerical data and generating high-quality visualizations of scalar and vector fields.

In this project, VTK was used to export the computed concentration fields at selected time steps in the `.vti` file format. Additionally, a `.pvd` file was generated to organize the individual output files into a time-dependent dataset, allowing the complete evolution of the solution to be visualized as an animation in ParaView. This facilitated the qualitative assessment of the numerical solution and enabled direct comparison with the provided reference solution.

The integration of VTK and ParaView into the simulation workflow simplified the post-processing stage and provided an effective means of validating the numerical implementation. The visualization tools were incorporated into the project's containerized development environments, enabling simulation results to be analyzed consistently across different systems.

2 Project Execution

2.1 Project Setup

The development of the numerical solver began with the establishment of a modular project structure to facilitate code organization, maintainability, and future extensions. The project was organized into dedicated directories for source files, header files, build configuration, scripts, simulation results, and documentation. This separation of responsibilities simplified the implementation process and enabled individual components of the solver to be developed and tested independently.

The build system was configured using CMake, while Git was used to initialize version control from the beginning of the project. A Docker environment was also prepared to provide a consistent development platform with all required dependencies pre-installed. This ensured that the project could be compiled and executed reliably across different systems without requiring additional software configuration.

The numerical implementation was divided into several modules, including the simulation configuration, sparse matrix assembly, numerical solvers, and visualization utilities. This modular architecture improved code readability and allowed new functionality, such as additional time integration schemes and output routines, to be incorporated without extensive modifications to the existing codebase.

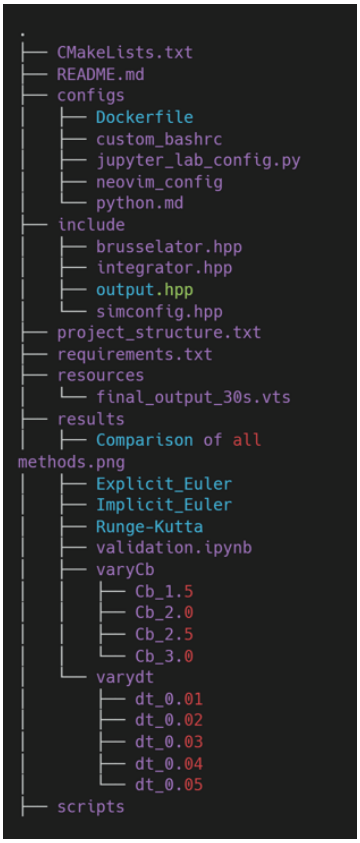


Figure 1: Directory structure of the project illustrating the modular organization of the source code, configuration files, scripts, and simulation results.

2.2 Laplacian Matrix Assembly

The first numerical component implemented in the solver was the spatial discretization of the diffusion operator. The two-dimensional computational domain was represented by a structured Cartesian grid, where each grid point stored the concentrations of the two reacting species. Spatial derivatives were approximated using the finite difference method with a five-point central

difference stencil, as illustrated in Figure 2. This approach provided a second-order accurate approximation of the Laplacian operator while maintaining a simple and efficient numerical implementation.

$$\nabla^2 C_{i,j} \approx \frac{C_{i+1,j} + C_{i-1,j} + C_{i,j+1} + C_{i,j-1} - 4C_{i,j}}{\Delta x^2}. \tag{1}$$

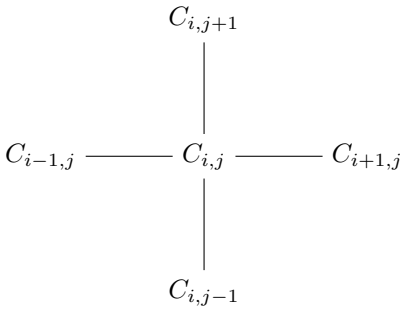


Figure 2: Five-point finite difference stencil used for approximating the Laplacian operator.

```cpp
Eigen::SparseMatrix<double> brusselator::buildA(const SimulationConfig&
{
    const int nx = config.nx;
    const int ny = config.ny;
    const int N  = nx * ny;
    const double alpha = 1.0 / (config.dx * config.dx);

    Eigen::SparseMatrix<double> A(N, N);

    std::vector<Eigen::Triplet<double>> triplets;
    triplets.reserve(5 * N);

    for (int i = 0; i < ny; ++i)
    {
        for (int j = 0; j < nx; ++j)
        {
            int k = i * nx + j;
            double diag = -4.0 * alpha;

            if (i == 0)
                diag += alpha;
            if (i == ny - 1)
                diag += alpha;
            if (j == 0)
                diag += alpha;
            if (j == nx - 1)
                diag += alpha;

            triplets.emplace_back(k, k, diag);

            if (i > 0)
            {
                int north = (i - 1) * nx + j;
                triplets.emplace_back(k, north, alpha);
            }
            if (i < ny - 1)
            {
                int south = (i + 1) * nx + j;
                triplets.emplace_back(k, south, alpha);
            }
            if (j < nx - 1)
            {
                int east = i * nx + (j + 1);
                triplets.emplace_back(k, east, alpha);
            }
            if (j > 0)
            {
                int west = i * nx + (j - 1);
                triplets.emplace_back(k, west, alpha);
            }
        }
    }

    A.setFromTriplets(triplets.begin(), triplets.end());

    return A;
}
```

Figure 3: Code snippet showing the Implementation of the sparse Laplacian matrix assembly for a single concentration field using Eigen triplets.

The implementation of the Laplacian matrix assembly is shown in Figure 3. The sparse matrix is constructed by looping over every grid point, computing the diagonal coefficient based on the homogeneous Neumann boundary conditions, and inserting only the non-zero neighbour contributions into an Eigen::SparseMatrix using the Eigen::Triplet data structure. This approach minimizes memory usage while preserving the five-point finite difference stencil.

Following the finite difference discretization, the diffusion operator was assembled into a sparse matrix representing the interactions between neighboring grid points. Since each interior cell depends only on its north, south, east, west, and central neighbors, only a small number of matrix entries are non-zero. To exploit this sparsity, the Laplacian matrix was stored using the `Eigen::SparseMatrix` class and assembled efficiently using triplet lists before conversion into compressed sparse format.

Special treatment was required for the computational boundaries, where homogeneous Neumann boundary conditions were imposed to enforce zero diffusive flux across the domain boundaries. Instead of introducing ghost cells, the values at the bound-

ary were reused during the finite difference discretization, ensuring that the normal concentration gradient remained zero while preserving the sparse structure of the system matrix.

Once assembled, the Laplacian matrix remained constant throughout the simulation and was constructed only once during the initialization phase. This matrix was subsequently reused by both the explicit and implicit time integration schemes, reducing computational overhead and improving the efficiency of the overall solver. The assembled Laplacian matrix formed the foundation of the numerical solver and was subsequently used by both the explicit and implicit time integration schemes described in the following sections.

2.3 Explicit Time Integration

After assembling the spatial discretization matrix, the resulting system of ordinary differential equations was advanced in time using an explicit integration scheme. The classical fourth-order Runge–Kutta (RK4) method was selected because of its improved temporal accuracy compared to Forward Euler. During development, the Forward Euler method was initially implemented to verify the correctness of the spatial discretization before extending the solver to RK4.

$$\mathbf{C}^{n+1} = \mathbf{C}^n + \frac{\Delta t}{6}\left(k_1 + 2k_2 + 2k_3 + k_4\right), \tag{2}$$

where k_1, k_2, k_3, and k_4 denote the four intermediate stage evaluations of the governing equations.

```cpp
Eigen::VectorXd integrate::rk4(
    const std::function<Eigen::VectorXd(const Eigen::VectorXd& C)>& f,
    const Eigen::VectorXd& C,
    const SimulationConfig& config
) {
    // ToDo
    Eigen::VectorXd k1=f(C);
    Eigen::VectorXd k2=f(C+config.dt*k1/2);
    Eigen::VectorXd k3=f(C+config.dt*k2/2);
    Eigen::VectorXd k4=f(C+config.dt*k3);

    return C+(config.dt/6)*(k1+2*k2+2*k3+k4);

  }
```

(a) RK4 implementation.

```cpp
Eigen::VectorXd brusselator::computeReaction(const Eigen::VectorXd& C,
 const SimulationConfig& config)
{
    //Allocate reaction vector
    Eigen::VectorXd reaction= Eigen::VectorXd::Zero(2*config.N);

    //Return immediately if reactions are disabled

    if (!config.reactionEnabled)
    {
        return reaction;
    }

    //Compute Brusselator source terms

    for (int i=0;i<config.N; i++)
    {
        double C1=C(i);
        double C2=C(i+config.N);

        reaction(i)=config.Ca+C1*C1*C2-config.Cb*C1-C1;

        reaction(i+config.N)=config.Cb*C1-C1*C1*C2;
    }

    return reaction;
}
```

(b) Reaction switch implementation.

Figure 4: Implementation of Runge Kutta solver (left), while the runtime reaction switch (right) enables or disables the nonlinear reaction source terms.

A runtime configuration flag was implemented to enable or disable the nonlinear reaction source terms. Disabling the reactions reduces the governing equations to a pure diffusion problem, which proved useful for verifying the spatial discretization against the provided reference solution.

2.4 Implicit Time Integration

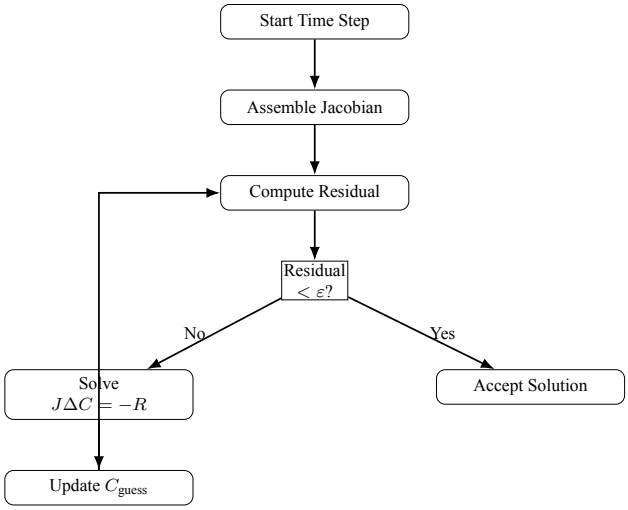
The implicit Backward Euler method was implemented to improve the numerical stability of the solver, particularly for larger time steps. Unlike the explicit Runge–Kutta method, the implicit formulation requires the solution of a nonlinear system at every time step. This was achieved using an inexact Newton method, where the Jacobian matrix was assembled once per time step and reused during the Newton iterations until convergence.

$$\mathbf{C}^{n+1} = \mathbf{C}^n + \Delta t\, \mathbf{f}(\mathbf{C}^{n+1}) \tag{3}$$

$$\mathbf{R}(\mathbf{C}_{\text{guess}}) = \mathbf{C}_{\text{guess}} - \mathbf{C}^n - \Delta t\left(\tilde{\mathbf{A}}\mathbf{C}_{\text{guess}} + \boldsymbol{\omega}(\mathbf{C}_{\text{guess}})\right) \tag{4}$$

where $\mathbf{C}_{\text{guess}}$ denotes the current Newton iterate, $\mathbf{C}^n$ is the solution from the previous time step, $\tilde{\mathbf{A}}$ represents the sparse diffusion matrix, and $\boldsymbol{\omega}$ contains the nonlinear reaction terms. The Newton iterations proceed until the residual norm falls below the prescribed convergence tolerance.

The Newton iterations are terminated once the Euclidean norm of the residual satisfies the convergence criterion



(a) Newton iteration algorithm used in the implicit Euler solver.

```cpp
//factorize once
Eigen::SparseLU<Eigen::SparseMatrix<double>> solver;

solver.compute(J);

const int maxNewtonIterations=20;
const double tolerance=1e-8;

for (int i=0; i<maxNewtonIterations;i++)
{
    Eigen::VectorXd residual=brusselator::computeResidual(
            Cguess, Cold,config);

    residualFile << timestep << "\t"
        << i << "\t" << residual.norm() << "\n";
    if(residual.norm() < tolerance)
    {
        break;
    }

    Eigen::VectorXd delta = solver.solve(-residual);

    if (solver.info() != Eigen::Success)
    {
        std::cerr << "Newton linear solve failed!" << std::endl;
        break;
    }

    Cguess+=delta;
}

return Cguess;
}
```

(b) Implementation of Newton loop.

```cpp
{
    //-----------------------------------------------------
    //Assembling of reaction Jacobian Jw
    //-----------------------------------------------------

    std::vector<Eigen::Triplet<double>> triplets;

    for (int cell=0; cell<config.N; cell++)
    {
        //global indices
        int c1=cell;
        int c2=cell+config.N;

        double C1=Cguess(c1);
        double C2=Cguess(c2);

        //-----------------------------------------------------
        //Local reaction Jacobian entries
        //-----------------------------------------------------

        double dw1_dC1= -(config.Ca+1.0)+ 2.0 * C1* C2;
        double dw1_dC2= C1*C2;

        double dw2_dC1= config.Cb - 2.0*C1*C2;
        double dw2_dC2= -C1*C1;

        //-----------------------------------------------------
        //Insert the 2x2 local block for one cell
        //-----------------------------------------------------

        triplets.emplace_back(c1,c1,dw1_dC1);
        triplets.emplace_back(c1,c2,dw1_dC2);

        triplets.emplace_back(c2,c1,dw2_dC1);
        triplets.emplace_back(c2,c2,dw2_dC2);
    }
```

(a) Jacobian Assembly.

```cpp
Eigen::VectorXd brusselator::computeResidual(const Eigen::VectorXd& Cguess, const Eigen::VectorXd& Cold, const SimulationConfig& config)
{
    //Evaluate the rhs which is the derivative of the
    //governing equation, that includes both diffusion and reaction
    Eigen::VectorXd rhs=
        brusselator::odeSystem(Cguess,config.A_tilde,config);

    // Compute the backward euler residual

    Eigen::VectorXd residual=
        Cguess - Cold - config.dt*rhs;

    return residual;

}
```

(b) Residual Implementation.

Figure 6: Assembly of Jacobian (left), and the residual Implementation (right)

$$\|\mathbf{R}(\mathbf{C}_{\text{guess}})\|_2 < 10^{-8}, \tag{5}$$

where the tolerance of 10^{-8} was chosen to ensure an accurate approximation of the implicit solution while avoiding unnecessary Newton iterations.

2.5 Output Generation and Visualization

To facilitate post-processing and verification, the numerical solution was periodically written to VTK image files (`.vti`) during the simulation. The output interval was specified by the user through the simulation configuration, allowing the solution to be stored only at selected time steps. In addition, a ParaView Data (`.pvd`) file was generated automatically to organize the time-dependent output sequence for visualization in ParaView.

To visualize the transient concentration fields, the numerical solution was periodically written to VTK ImageData (`.vti`) files. At every user-defined output interval, the concentration vectors were converted into VTK data structures and stored as individual files. Each file therefore represents the complete spatial solution at a single simulation time.

Although each `.vti` file contains the solution at one instant in time, ParaView requires additional information to recognize the entire sequence as a transient simulation. Loading dozens of individual VTK files manually would be inefficient and would not automatically preserve the temporal ordering of the solution.

```cpp
auto grid= vtkSmartPointer<vtkImageData>::New();
//New() creates an object of type vtkImageData somewhere
//inside heap, and grid is a vtkSmartPointer which points
//to this object

grid->SetDimensions(config.nx,config.ny,1); //since the problem is
2D
// we use z=1 & 10 points in x and y directions

grid->SetSpacing(config.dx,config.dy,1);

//Create vtkDoubleArray. Just an empty table that stores
//the values of concentrations
auto C1array= vtkSmartPointer<vtkDoubleArray>::New();

auto C2array= vtkSmartPointer<vtkDoubleArray>::New();
```

(a) code for creating vtkImage.

```cpp
for(int l=0;l<N;l++)
{
    C2array->SetValue(l,C(N+l));
}

grid->GetPointData()->AddArray(C2array);
grid->GetPointData()->SetActiveScalars("C1");

std::ostringstream filename; //empty string
//append pieces to it

filename << config.outputDirectory <<"/output_";

//append timestep

//format it with leading zeros
filename << std::setw(4) << std::setfill('0') << timestep;
// std::setw(4) means to use atleast 4 characters
// std::setfill('0') is used so that if the number
// is shorter than 4 characters, fill the remaining places
// with zeros

filename << ".vti";

auto writer=vtkSmartPointer<vtkXMLImageDataWriter>::New();

writer->SetFileName(filename.str().c_str());
//filename.str() is returns a std::string, .c_str()
//converts that into the C-style string (const char*) that
//VTK expects

writer->SetInputData(grid);

writer->Write(); //actually creates the file on disk
```

(b) code for writing vtk files.

Figure 7: Implementation of the VTK output routine used to write the concentration fields for visualization.

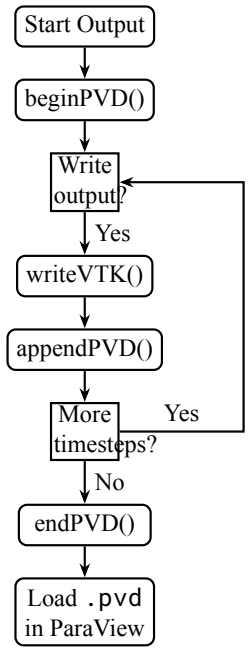


Figure 8: Workflow for generating the VTK output files and constructing the PVD file for transient visualization in ParaView.

Although the VTK (`.vti`) files store the concentration fields at individual time steps, they do not contain information about the temporal relationship between successive outputs. To enable time-dependent visualization in ParaView, a ParaView Data (`.pvd`) file was generated alongside the VTK files. The implementation was divided into three functions: `beginPVD()`, `appendPVD()`, and `endPVD()`. The `beginPVD()` function initializes the XML structure of the PVD file before the simulation starts, `appendPVD()` is called after every VTK output to record the simulation time and the corresponding VTK filename, and `endPVD()` finalizes the XML document once the simulation is complete. This modular implementation ensures that the generated PVD file correctly references every output snapshot, allowing the complete transient simulation to be loaded and animated directly in ParaView.

```cpp
void beginPVD()
{
    std::ofstream pvd("solution.pvd");

    pvd << "<?xml version=\"1.0\"?>\n";

    pvd << "<VTKFile type=\"Collection\" ve

    pvd << "  <Collection>\n";
}
```

(a) Code to open the pvd.

```cpp
{
    //open solution.pvd in append mode
    std::ofstream pvd("solution.pvd", std::ios::app);

    //Build the VTK filename
    std::ostringstream filename;

    filename << config.outputDirectory <<"/output_" << std::setw(4) <<
std::setfill('0')
        << timestep << ".vti";

    // Write one DataSet entry
    pvd << "  <DataSet " << "timestep=\"" << time << "\" "
        << "file=\"" << filename.str() << "\"/>\n";
```

(b) Code for writing indexing files.

```cpp
void endPVD()
{
    std::ofstream pvd("solution.pvd", std::ios::app);

    pvd << " </Collection>\n";

    pvd << "</VTKFile>\n";
}
```

(c) Code for end PVD file.

Figure 9: code for closing the pvd file.

2.6 Bash Automation

A Bash automation script was developed to simplify the execution of multiple simulations and the required parameter studies. The script accepts user-defined command-line arguments specifying the parameter to be varied and the corresponding values. For each parameter set, the script automatically updates the simulation configuration, builds the solver, executes the simulation, and stores the generated output in a dedicated results directory. This automated workflow eliminates repetitive manual steps and ensures a reproducible organization of simulation data.

Parameter Variation 1

The first parameter study investigates the influence of the Brusselator coefficient C_b while keeping $C_a = 1$ fixed. The script automatically executes the explicit Runge–Kutta solver for

$$C_b = \{1.5,\ 2.0,\ 2.5,\ 3.0\}.$$

For each simulation, a separate output directory is created containing the generated VTK files, the corresponding PVD file, and the automatically generated start script. This organization facilitates direct comparison of the concentration fields obtained for different reaction parameters.

Parameter Variation 2

The second parameter study evaluates the convergence behaviour of the implicit Euler solver by varying the time step while keeping the remaining simulation parameters fixed. The Bash script automatically performs simulations for

$$\Delta t = \{0.01,\ 0.02,\ 0.03,\ 0.04,\ 0.05\}\ \text{s}.$$

During each simulation, the total number of Newton iterations is recorded and written to the output files. These data are later used to analyse the influence of the time-step size on the convergence behaviour of the implicit solver.

The automation framework enables both required parameter studies to be executed using a single command. By automatically generating the simulation directories and storing all associated output files for each parameter set, the script provides a reproducible and well-organized workflow for conducting numerical experiments and comparing simulation results.

```
# Loop to go through the values which the user gave
for value in "${values[@]}"

# assign values that usr gave one at a time to Cb
do
    if [ "$parameter" = "Cb" ]
    then
        Cb=$value
    fi

    if [ "$parameter" = "dt" ]
    then
        dt=$value
    fi


# Generate start.sh
    cat > start.sh << EOF
#!/bin/bash

../build/brusselator $solver $nx $ny $endtime $dt $writeInterval $Ca $
Cb
EOF
```

(a) script for Cb parameter variation.

```
# Loop to go through the values which the user gave
for value in "${values[@]}"

# assign values that usr gave one at a time to Cb
do

    if [ "$parameter" = "dt" ]
    then
        dt=$value
    fi


# Generate start.sh
    cat > start.sh << EOF
#!/bin/bash

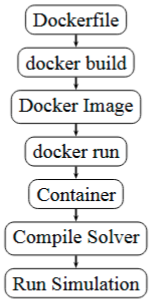
../build/brusselator $solver $nx $ny $endtime $dt $writeInterval $Ca $
Cb
EOF
```

(b) script for dt parameter variation.

Figure 10: Implementation of bash scripting for automating the parametric variation using a single command.

2.7 Docker Environment

To ensure that the project could be built and executed consistently across different systems, the solver was containerized using Docker. The Docker image provides a self-contained Ubuntu environment with all required libraries and development tools pre-installed, eliminating dependency and compatibility issues between host systems. This enables reproducible compilation and execution of the solver independent of the local machine configuration.



(a) Workflow of the Docker environment used to build and execute.

```
# Base on minimal Debian image
FROM debian:stable-slim

# Avoid interactive prompts during package installation
ENV DEBIAN_FRONTEND=noninteractive

# install tools
RUN apt-get update && apt-get install -y \
    gcc \
    g++ \
    make \
    cmake \
    gdb \
    git \
    neovim \
    libboost-all-dev \
    python3.13-venv \
    tree \
    paraview \
    libxcb-cursor0 \
    libeigen3-dev \
    libvtk9-dev \
    && apt-get clean \
    && rm -rf /var/lib/apt/lists/*

# Set working directory
WORKDIR /workspace

# add custom configs
COPY configs/custom_bashrc /root/.bashrc
COPY configs/neovim_config /root/.config/nvim/init.lua
COPY configs/jupyter_lab_config.py /root/.jupyter/

# Open bash shell
CMD ["/bin/bash"]
```

(b) Dockerfile.

2.8 Challenges Encountered and Solutions

2.8.1 Challenge 1 - Sparse Laplacian Assembly

During the implementation of the finite-difference discretization, correctly assembling the sparse Laplacian matrix proved challenging. Particular attention was required when converting the two-dimensional computational grid into a one-dimensional state vector while simultaneously enforcing homogeneous Neumann boundary conditions.

2.8.2 Solution

The Laplacian matrix was assembled using Eigen's sparse matrix representation together with triplet lists. The mapping between two-dimensional grid indices and the corresponding vector indices was carefully verified, while the boundary conditions were incorporated by modifying the stencil coefficients at the domain boundaries. The implementation was validated by comparing the diffusion-only solution with the provided reference solution.

2.8.3 Challenge 2 - Implicit Euler and Newton Method

The implementation of the implicit Euler solver represented the most computationally demanding part of the project. Formulating the nonlinear residual, assembling the Jacobian matrix, and ensuring stable convergence of the Newton iterations required careful debugging and verification.

2.8.4 Solution

The governing equations were reformulated as a nonlinear residual equation, which was solved using an inexact Newton method. A sparse Jacobian matrix was assembled once per time step and solved using Eigen's sparse linear solver. Convergence was monitored through the residual norm and terminated once the prescribed tolerance was satisfied.

2.8.5 Challenge 3 - Docker and Dependencies

Setting up a reproducible software environment presented several practical challenges, particularly with the installation of external libraries such as Eigen and VTK and ensuring compatibility across different systems.

2.8.6 Solution

A Docker container was created containing all required libraries and development tools. The containerized environment ensured identical build and execution behaviour independent of the host operating system, thereby improving portability and reproducibility.

2.8.7 Challenge 4 - ParaView in Docker

Visualizing the generated VTK files inside the Docker container initially resulted in graphical interface errors because the container was unable to access the host display server.

2.8.8 Solution

The issue was resolved by configuring graphical forwarding between the Docker container and the host operating system. Once the display environment was correctly configured, ParaView was able to load and visualize the generated VTK files successfully.

2.8.9 Challenge 5 - Automation

Executing the required parameter studies manually involved repeatedly modifying the simulation parameters, recompiling the project, and organizing the generated output files, making the workflow time-consuming and error-prone.

2.8.10 Solution

A Bash automation script was developed to perform the complete workflow automatically. The script generated the required simulation configurations, executed the solver for each parameter set, and stored the generated output files in dedicated directories, enabling efficient execution of both parameter studies.

2.9 Lessons Learned and Future Improvements

This project provided valuable experience in both numerical simulation and scientific software development. Beyond implementing the numerical methods required for solving the Brusselator reaction–diffusion system, the project emphasized the importance of software design, reproducibility, and automation in scientific computing. Several lessons were learned throughout the development process, while a number of opportunities for further improvement were identified.

The project reinforced the importance of developing numerical software in a structured and incremental manner. Implementing and validating the explicit Runge–Kutta solver before introducing the implicit Euler method simplified debugging and increased confidence in the correctness of the spatial discretization. Furthermore, the use of sparse matrix representations significantly reduced memory usage and computational cost, highlighting the importance of selecting appropriate data structures for large numerical problems.

Another important lesson was the value of modern software engineering practices in scientific computing. The use of CMake for project organization, Docker for a reproducible development environment, GitLab for version control, and Bash scripting for automation resulted in a workflow that was considerably more robust and easier to reproduce than manual execution.

Several improvements could be incorporated into the current implementation. The implicit solver could be extended with adaptive time stepping, allowing the time step size to be adjusted automatically based on the convergence behaviour of the Newton iterations. This would improve computational efficiency while maintaining numerical accuracy.

The numerical performance could also be enhanced through parallelization using OpenMP or MPI, enabling simulations on larger computational grids. Additionally, more advanced iterative sparse linear solvers and preconditioning techniques could further reduce the computational cost of the implicit solution procedure.

From a software perspective, the simulation parameters could be read from an external configuration file rather than requiring recompilation when default values are modified. This would further simplify parameter studies and improve the flexibility of the software.

Finally, the solver could be extended to support additional reaction–diffusion models, more complex boundary conditions, and adaptive mesh refinement, making the framework applicable to a wider range of scientific and engineering problems.

Overall, the project demonstrated that successful scientific computing requires a combination of sound numerical methods and well-designed software engineering practices. The experience gained throughout this project provides a strong foundation for developing larger and more sophisticated computational simulation tools in future research and engineering applications.

2.10 Results

This chapter presents the numerical results obtained using the developed Brusselator solver. The implementation was first validated against the supplied reference solution for the diffusion-only case. Subsequently, the influence of selected simulation parameters was investigated through the required parameter studies. Finally, quantitative error metrics are presented to assess the accuracy of the numerical implementation.

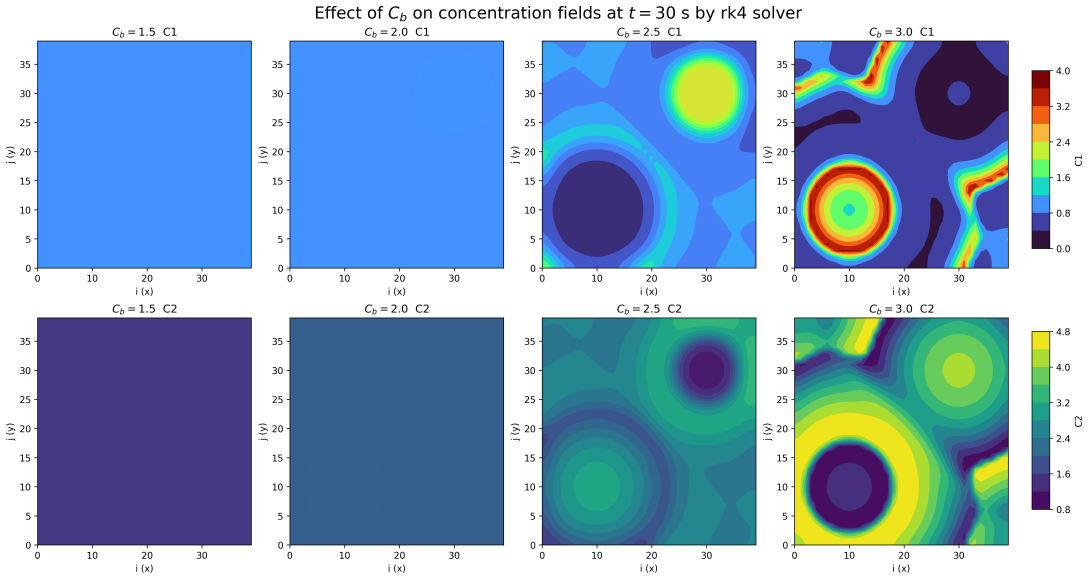


Figure 12: Effect of Cb variation on the concentration field.

2.11 Effect of the Model Parameter C_b

Figure 12 presents the concentration distribution obtained for different values of the model parameter C_b. As the value of C_b increases, noticeable changes in the spatial concentration field are observed. The reaction kinetics become increasingly dominant, resulting in stronger concentration gradients and more pronounced spatial structures.

For lower values of C_b, diffusion dominates the transport process, producing comparatively smooth concentration profiles. Increasing C_b enhances the nonlinear reaction term, leading to greater local production of species and consequently sharper concentration variations throughout the computational domain.

The simulations demonstrate the expected sensitivity of the Brusselator system to changes in the reaction parameter, confirming that the implemented reaction terms correctly reproduce the characteristic behaviour of the model.

2.12 Validation with the Reference Solution

To verify the correctness of the numerical implementation, the diffusion-only solver was compared with the reference solution provided for the final simulation time. Figure **??** illustrates the close agreement between the computed solution and the reference concentration field.

Both solutions exhibit nearly identical spatial distributions, with only minor numerical differences visible near regions of relatively high concentration gradients. The overall agreement confirms that the spatial discretization, boundary condition implementation, and time integration scheme were implemented correctly.

This comparison provides confidence that the solver accurately reproduces the expected numerical solution before enabling the nonlinear reaction terms.

2.13 Effect of Time Step Size

The influence of the time-step size was investigated using the implicit Euler solver. Figure 14 compares the numerical solutions obtained using different values of Δt.

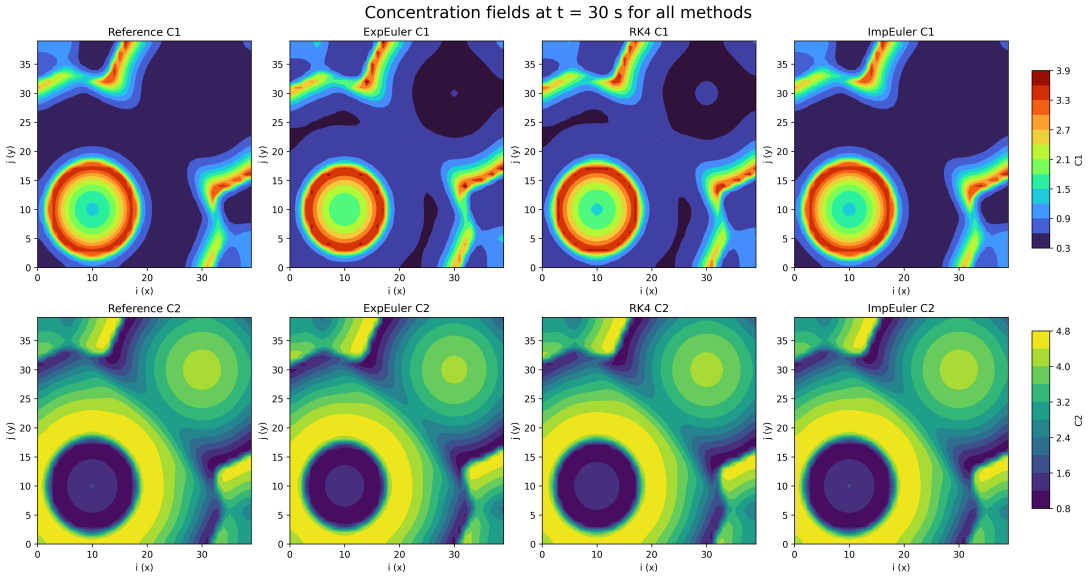


Figure 13: Comparison of all methods with same parameters at final timstep.

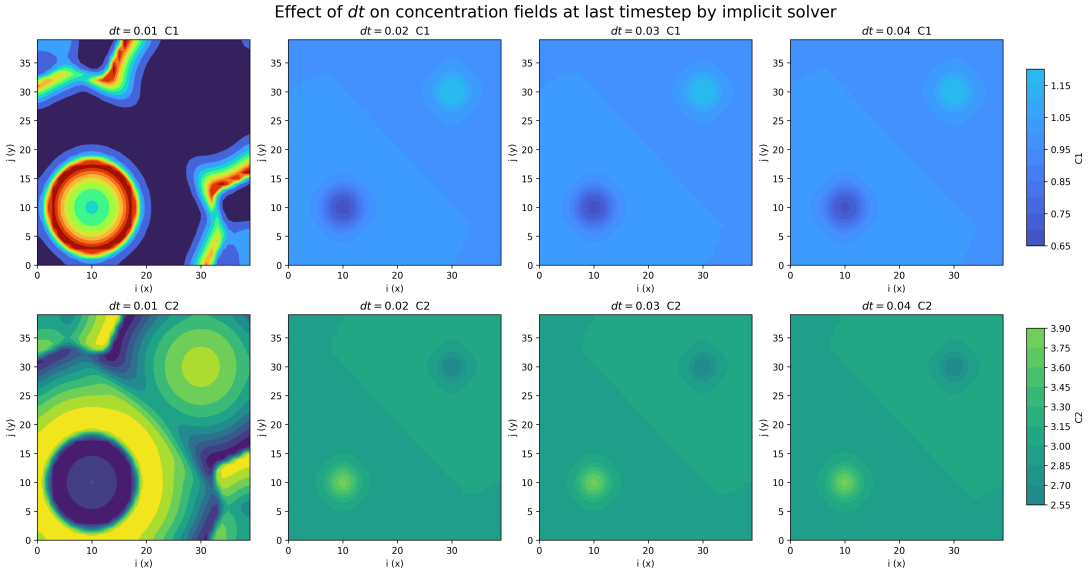


Figure 14: Effect of varying timestep size for the implicit solver.

For the range of time steps considered, the computed concentration fields remain qualitatively similar, indicating that the implicit Euler method maintains numerical stability even for relatively large time-step sizes. However, increasing the time step introduces additional numerical diffusion, resulting in slightly smoother concentration distributions.

Smaller time steps preserve the spatial features of the solution more accurately but require a larger number of time integrations and Newton iterations, thereby increasing the computational cost. This behaviour illustrates the trade-off between computational efficiency and numerical accuracy.

2.14 Quantitative Error Analysis

2.15 Effect of Time Step Size

	Method	L2(C1)	L∞(C2)	MAE(C1)	L2(C2)	MAE(C2)
0	Explicit Euler	3.28878251e-08	2.37882991e-07	1.00451326e+00	9.09798801e-08	3.11865783e+00
1	RK4	2.24190783e-01	1.85679578e+00	1.01049268e+00	2.69746044e-01	3.11203790e+00
2	Implicit Euler	4.18817437e-01	2.85815884e+00	1.01711535e+00	5.07891849e-01	3.10411930e+00

Figure 15: This table shows the various error metrics obtained for Explicit Euler, Runge-kutta and Implicit Euler.

To complement the visual comparison, quantitative error metrics were computed by comparing the numerical solution with the supplied reference solution. Figure 15 summarizes the calculated error measures.

All reported error values remain relatively small, indicating that the implemented solver accurately reproduces the reference solution. The consistency between the visual comparison and the quantitative error metrics confirms the correctness of both the finite-difference spatial discretization and the implemented time integration methods.

The low error values further demonstrate that the developed solver provides a reliable numerical approximation of the governing reaction–diffusion equations.

2.16 Discussions

The numerical experiments demonstrate that the developed solver successfully captures the expected behaviour of the Brusselator reaction–diffusion system. The parameter studies confirm the sensitivity of the concentration field to changes in both the reaction parameter C_b and the numerical time-step size.

The comparison with the supplied reference solution shows excellent agreement, indicating that the implementation of the finite-difference discretization, homogeneous Neumann boundary conditions, and time integration schemes is correct. The accompanying error analysis further supports this conclusion by showing only small deviations from the reference solution.

The implicit Euler method exhibited stable behaviour over the investigated range of time-step sizes, while the explicit solver accurately reproduced the reference solution when an appropriate time step was selected. Together, these results demonstrate that the developed implementation is both accurate and robust for the considered benchmark problem.

2.17 Conclusion

This project successfully implemented a numerical solver for the two-dimensional Brusselator reaction–diffusion system using finite-difference spatial discretization. Both an explicit fourth-order Runge–Kutta method and an implicit Euler method were developed, together with the necessary software infrastructure for automated parameter studies, containerized execution, and continuous integration.

The explicit solver produced accurate results and showed excellent agreement with the provided reference solution for the diffusion-only validation case. The parameter study investigating the influence of the reaction parameter C_b demonstrated the expected changes in the concentration field, confirming the correct implementation of the reaction kinetics.

Although the implicit Euler solver was successfully implemented and remained numerically stable, the parameter study involving variations in the time-step size did not produce results of the same quality as the explicit solver. As the time step increased, noticeable deviations from the expected solution were observed, indicating reduced numerical accuracy. These discrepancies may be attributed to the first-order temporal accuracy of the implicit Euler method, the chosen Newton solver parameters, or

the absence of adaptive time stepping. Consequently, further refinement of the nonlinear solution procedure and convergence strategy would likely improve the performance of the implicit solver.

Overall, the project achieved its primary objectives by developing a functional and reproducible numerical framework for solving the Brusselator equations. Beyond the numerical implementation, the project also demonstrated the importance of software engineering practices such as modular code design, automation, version control, Docker-based reproducibility, and continuous integration. Future work could focus on improving the accuracy and efficiency of the implicit solver through adaptive time stepping, more advanced nonlinear solvers, improved preconditioning techniques, and parallelization for larger computational problems.